1. Multi-issue Tomasulo implementation

Ali Alfageeh
October 15, 2026

The task was to extend the supplied Python simulator so it could issue between one and four instructions per cycle, in program order. The completed version keeps the ISSUE, EX, MEM, WB and COMMIT modules. It also handles dependencies within one issue bundle, branch recovery and loads that run before an older store's address is known.

Issue width is read from the input file and defaults to 1. The command line can override it. Only values 1 through 4 are accepted. Changing the width changes how quickly instructions enter the machine; it does not change the program's final result.

What the simulator supports

All ten instructions in the handout are supported: Ld, Sd, Beq, Bne, Add, Sub, Addi, Add.d, Sub.d and Mult.d. Hardware settings include reservation-station counts, the ROB capacity, functional-unit counts, execution latencies and memory latency. R0 stays zero.

Integer and address units are unpipelined. FP add and multiply units are pipelined. Loads and stores share one unpipelined memory port, and results share one common data bus. Retirement is limited to one instruction per cycle in this implementation.

Main findings

All 36 benchmark runs, covering nine programs at four issue widths, produce the same architectural state as a separate sequential interpreter. The test suite also passes 20 tests, including 160 randomized program runs across the four widths.

Wider issue helps some programs more than others. The late-critical-path program falls from 27 to 21 cycles and the branch program from 25 to 20. The independent program stays at 20 cycles because issuing more work cannot remove its later bottleneck. Section 4 gives all cycle counts, and the appendices show both supplied sample cases at every issue width.

The supplied sample report also widens writeback and retirement. This implementation keeps a single CDB, as specified in the handout, and a one-instruction retirement limit. Its timing tables therefore should not be expected to match the sample report's numbers.

2. Changes to the supplied code

File	Change
init.py / model.py	Parse width and hardware settings; represent instructions, operands and entries with separate dataclass objects
issue.py	Issue an in-order bundle, capture operands and rename destinations with unique tags
exe.py	Schedule ready instructions, track completion cycles, resolve branches and detect memory-order violations
mem.py	Forward from older stores and serialize accesses through the memory port
wb.py / commit.py	Broadcast one result, wake matching consumers and retire the ROB head
simulator.py	Step the stages and recover the RAT, queues and executing work after a squash
main.py / print_status.py	Provide the CLI, timing table, complete register values, nonzero memory and JSON output

Issuing a bundle in program order

The issue stage tries up to the configured number of instructions. Before allocating each one, it checks the ROB and the required reservation station or load/store queue. If the next instruction has no space, the bundle stops there. A younger instruction cannot skip past it simply because another unit is free.

Renaming within the same cycle

Source operands are captured before the destination updates the RAT. Later instructions in the bundle then see the earlier instruction's rename. Each dynamic instruction has a new tag, so reusing a register or ROB position cannot wake a consumer with an unrelated result.

```
Addi R1 R0 7
Add  R2 R1 R1
```

At issue width 2, both instructions can enter in cycle 1. The second waits for the first: Addi executes in cycle 2, writes back in 3 and commits in 4. Add executes in cycle 4, writes back in 5 and commits in 6. The final value of R2 is 14. The test suite checks these exact cycles.

3. Scheduling, recovery and memory

Cycle accounting

Each cycle calls COMMIT, WB, MEM, EX and ISSUE in that order. Explicit timestamps prevent an instruction from executing in its issue cycle or a consumer from using a result in the cycle that result is broadcast. EX and MEM ranges include both endpoints. A one-cycle EX operation that starts in cycle 2 finishes in cycle 2.

The oldest ready instructions are selected separately for each functional-unit type. FP units can accept new work each cycle; integer and address units must finish the current operation first. Results may finish out of order, but only the ROB head can change architectural state at retirement.

Branch prediction and recovery

The BTB has eight tagged entries indexed by the low three bits of the word PC. Each entry holds a target and a one-bit outcome. Missing entries predict not taken. Beq and Bne resolve in EX. A wrong prediction removes younger ROB entries, reservation stations, queue entries and executing work, then rebuilds the RAT from surviving writers. If the branch resolves in cycle n, correct fetching resumes in n+2.

Stores, forwarding and speculative loads

Stores calculate their address without waiting for store data, but write memory only at the head of the ROB. A load searches for the youngest older store at the same address. It forwards in one cycle if that store's data is ready, and waits otherwise. A nonmatching store between them does not block the matching store from forwarding.

A load may read memory while an older store's address is still unknown. If that store later resolves to the same address, the load and all younger work are squashed and replayed. The check includes loads that have left the load/store queue but are still in the ROB, so an old speculative value cannot retire unnoticed.

Input conventions

Addresses are bytes, with 64 aligned words at addresses 0 through 252. Branch offsets are signed words: target = PC + 4 + offset * 4. The bundled eight-instruction vector loop uses -8 to return to its first instruction. The starter's legacy -12 immediate was corrected for this convention. The parser accepts case-insensitive instructions, optional operand commas and comments.

4. Results across issue widths

The table reports simulated cycles, not elapsed runtime. Each row uses the same input and hardware settings at all four widths. Speedup is width-1 cycles divided by width-4 cycles. All final registers, memory values and committed instruction sequences match the sequential interpreter.

Benchmark	W1	W2	W3	W4	Speedup
branches	25	20	20	20	1.25x
dependencies	20	20	20	20	1.00x
independent	20	20	20	20	1.00x
late_critical_path	27	23	21	21	1.29x
memory	42	41	40	40	1.05x
sample_report_01	36	36	36	36	1.00x
sample_report_02	47	46	46	46	1.02x
structural	40	40	40	40	1.00x
vector_loop	66	65	65	65	1.02x

Source: results/summary.csv, checked against 36 fresh simulator runs while building this PDF. Complete timing traces are in results/.

Why the speedups are small

The late-critical-path case benefits because the wider issue stage reaches a long multiply/add chain sooner. Most of its gain is already present at width 3: both widths 3 and 4 take 21 cycles. The branch benchmark reaches 20 cycles at width 2 and gains nothing from wider issue after that.

The independent benchmark remains at 20 cycles. Its early multiply delays the head of the ROB, and retirement then releases only one instruction per cycle. The dependency benchmark is limited by dependent work. In the structural benchmark, small queues prevent the front end from keeping a wider bundle full.

The memory test improves from 42 to 40 cycles, but its one memory port and replay still cost time. The vector loop drops only from 66 to 65. These results are consistent with changing issue width while leaving the CDB, memory port and retirement width fixed.

The two sample-report inputs take 36/36/36/36 and 47/46/46/46 cycles. Their complete instruction tables and final values are included in Appendices 1 and 2.

5. Additional test cases and correctness

The extra benchmark programs check behavior that the two supplied arithmetic streams do not cover. Each program runs at widths 1, 2, 3 and 4. The values below are selected final-state checks, not a substitute for comparing all registers and memory.

Benchmark	Purpose and checked result
branches	Loop prediction and wrong-path stores; R2=4, R3=8 and memory remains zero
dependencies	RAW, WAR, WAW and R0; F3=4, F4=7, R3=21 and R0=0
independent	Mixed ready work across functional units; F3=6, F7=9, F10=6 and R4=4
late_critical_path	A late multiply/add chain; F3=6 and F4=12
memory	Unknown-store replay and forwarding; F3=9, F4=18, F5=7; Mem[0]=9, Mem[4]=7, Mem[8]=9
structural	Two-entry ROB and one-entry queues; F7=16 and Mem[0]=8
vector_loop	Three loop iterations; R1=0, R2=20; Mem[24]=15, Mem[28]=11, Mem[32]=7

Checks beyond the benchmark totals

The 20 unittest tests check hand-calculated timings, dependencies within a bundle, blocked issue, tiny ROB/queue capacities, FU counts and pipelining, CDB contention, store-data dependencies, precise faults, BTB collisions, parser errors and CLI behavior. They also check that retirement remains ordered and that no cycle broadcasts two results.

Forty deterministic randomized programs run at each width, for 160 differential runs. They vary capacities, latencies and functional-unit counts and mix arithmetic, memory operations and forward branches. Separate targeted tests exercise backward loops and loads that must be replayed after an older store resolves.

The reference interpreter models the instruction semantics without reservation stations, renaming or timing. Matching it checks architectural results independently of the scheduling model. Hand-calculated tests are still needed for cycle timing, because the sequential interpreter has no expected cycle count.

6. What works and remaining limitations

The required instructions, all four issue widths, configurable hardware, register renaming, in-order retirement, branch recovery, store forwarding and memory-order replay work in the tests described here. No known failing case remains in that tested scope. Passing those tests does not establish that every possible program or hardware configuration is correct.

Arithmetic and exceptions

This is a timing and instruction-order simulator. It uses Python floating-point numbers and unbounded integers, so it does not reproduce IEEE-754 single-precision rounding or integer overflow. Invalid or unaligned dynamic memory accesses raise an error at retirement; a fault on a squashed path can be discarded. General exception handlers, interrupts, caches and virtual memory are not modeled.

Prediction and replay

The predictor updates at EX and is not rolled back if an older branch later squashes that branch. This can affect future predictions and timing, although the ROB still protects architectural state. Memory replay is conservative: a load can be replayed even when an intervening store would have shielded it from an older store.

Forwarding occupies the modeled memory-stage port for one cycle. Extra load/store functional units add address calculators, not extra memory ports. Fetch has no alignment or bundle restrictions beyond issue width. Keeping every dynamic instruction in the output history also means memory use grows with program length.

Input and sample-report differences

The handout's short syntax example uses R1=10 with an 8(R1) access. Address 18 is unaligned under the 64-word memory model used here; an executable test should use an aligned base. This restriction is reported explicitly rather than silently rounding the address.

The supplied sample report uses different assumptions for writeback, retirement and memory concurrency, and lists unsupported branches as a limitation. Its instruction streams are useful tests, but its cycle counts are not an oracle for this model. Branches are supported and tested in the submitted implementation.

7. Running the simulator and sources

The simulator requires Python 3.9 or newer and no third-party packages. From the repository root:

```
cd Assignment1/Tomasulo
python3 tomasulo-python/code/main.py \
  benchmarks/vector_loop.txt --issue-width 4
python3 -m unittest discover -s tests -v
python3 run_benchmarks.py
```

The CLI prints ISSUE, EX, MEM, WB and COMMIT for every dynamic instruction, including explicitly marked squashed work. It prints all integer and FP registers and nonzero memory values. JSON and output-file options are also available.

The benchmark runner regenerates the 36 traces, summary.csv, summary.md and final_states.json in results/. To rebuild this PDF and the BP report after running the benchmarks, use the following commands from the repository root:

```
python3 -m venv .report-venv
.report-venv/bin/pip install -r requirements-report.txt
.report-venv/bin/python scripts/build_assignment_reports.py
```

The PDF builder reruns all 36 Tomasulo cases and compares them with the sequential interpreter and saved output files. It stops if a saved result is stale. It also checks the BP counts and source checksums before using those measurements.

Sources and attribution

1. *Assignment: Tomasulo Algorithm Implementation*, supplied four-page handout. The issue-width requirement, ISA, BTB, memory organization and recovery timing come from this specification.

2. Supplied *Sample Tomasulo Assignment Report*. Its two test streams and hardware configurations are retained in the appendices. Every timing table in this report is produced by the submitted simulator.

3. Supplied tomasulo-python starter, credited in its source to Qing, May 2017. The stage module organization is retained, with the changes described in Section 2.

Codex assisted with implementation, testing and report preparation. All numerical tables in this report were checked against the included code and output files.

Repository: fqeh/CompArchAssignment1 / Assignment1/Tomasulo

Appendix 1. Sample test case 01

This input comes from the supplied sample report. Its hardware configuration and instruction stream are retained. Each run changes only the issue width.

Unit	RS entries	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	4	3	-	1
FP multiplier	4	5	-	1
Load/store	4	1	6	1

ROB entries: 64. **Issue widths:** 1, 2, 3 and 4.
Initial registers: R1=12, R2=32, F20=3; all others are zero.
Initial memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4; all other words are zero.

```
 1. Ld F1 0(R1)
 2. Mult.d F2 F1 F0
 3. Add.d F3 F3 F2
 4. Mult.d F2 F1 F1
 5. Add.d F4 F2 F4
 6. Addi R1 R1 8
 7. Ld F1 0(R1)
 8. Mult.d F2 F1 F0
 9. Add.d F3 F3 F2
10. Mult.d F2 F1 F1
11. Add.d F4 F2 F4
12. Addi R1 R1 8
13. Ld F1 0(R1)
14. Mult.d F2 F1 F0
15. Add.d F3 F3 F2
16. Mult.d F2 F1 F1
17. Add.d F4 F2 F4
```

Final values at every issue width

Nonzero registers: R1=28, R2=32, F1=5, F2=25, F4=26, F20=3.

Nonzero memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All other register and memory values are zero.

Input: benchmarks/sample_report_01.txt. Complete outputs: results/sample_report_01_w1.txt through sample_report_01_w4.txt.

Case 01: issue widths 1 and 2

EX and MEM intervals include both endpoints. A dash means the stage does not apply. All 17 instructions commit; these sample streams contain no branches.

Issue width 1 | 36 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	2	10-14	-	15	16
Add.d F3 F3 F2	3	16-18	-	19	20
Mult.d F2 F1 F1	4	11-15	-	16	21
Add.d F4 F2 F4	5	17-19	-	20	22
Addi R1 R1 8	6	7	-	8	23
Ld F1 0(R1)	7	9	10-15	17	24
Mult.d F2 F1 F0	8	18-22	-	23	25
Add.d F3 F3 F2	9	24-26	-	27	28
Mult.d F2 F1 F1	10	19-23	-	24	29
Add.d F4 F2 F4	11	25-27	-	28	30
Addi R1 R1 8	12	13	-	14	31
Ld F1 0(R1)	13	15	16-21	22	32
Mult.d F2 F1 F0	14	23-27	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	17	24-28	-	30	35
Add.d F4 F2 F4	18	31-33	-	34	36

Issue width 2 | 36 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	1	10-14	-	15	16
Add.d F3 F3 F2	2	16-18	-	19	20
Mult.d F2 F1 F1	2	11-15	-	16	21
Add.d F4 F2 F4	3	17-19	-	20	22
Addi R1 R1 8	3	4	-	5	23
Ld F1 0(R1)	4	6	9-14	17	24
Mult.d F2 F1 F0	4	18-22	-	23	25
Add.d F3 F3 F2	5	24-26	-	27	28
Mult.d F2 F1 F1	5	19-23	-	24	29
Add.d F4 F2 F4	6	25-27	-	28	30
Addi R1 R1 8	6	7	-	8	31
Ld F1 0(R1)	7	9	15-20	21	32
Mult.d F2 F1 F0	10	22-26	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	16	23-27	-	30	35
Add.d F4 F2 F4	17	31-33	-	34	36

Case 01: issue widths 3 and 4

EX and MEM intervals include both endpoints. A dash means the stage does not apply. All 17 instructions commit; these sample streams contain no branches.

Issue width 3 | 36 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	1	10-14	-	15	16
Add.d F3 F3 F2	1	16-18	-	19	20
Mult.d F2 F1 F1	2	11-15	-	16	21
Add.d F4 F2 F4	2	17-19	-	20	22
Addi R1 R1 8	2	3	-	4	23
Ld F1 0(R1)	3	5	9-14	17	24
Mult.d F2 F1 F0	3	18-22	-	23	25
Add.d F3 F3 F2	3	24-26	-	27	28
Mult.d F2 F1 F1	4	19-23	-	24	29
Add.d F4 F2 F4	4	25-27	-	28	30
Addi R1 R1 8	4	5	-	6	31
Ld F1 0(R1)	5	7	15-20	21	32
Mult.d F2 F1 F0	10	22-26	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	16	23-27	-	30	35
Add.d F4 F2 F4	17	31-33	-	34	36

Issue width 4 | 36 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	1	10-14	-	15	16
Add.d F3 F3 F2	1	16-18	-	19	20
Mult.d F2 F1 F1	1	11-15	-	16	21
Add.d F4 F2 F4	2	17-19	-	20	22
Addi R1 R1 8	2	3	-	4	23
Ld F1 0(R1)	2	5	9-14	17	24
Mult.d F2 F1 F0	2	18-22	-	23	25
Add.d F3 F3 F2	3	24-26	-	27	28
Mult.d F2 F1 F1	3	19-23	-	24	29
Add.d F4 F2 F4	3	25-27	-	28	30
Addi R1 R1 8	3	5	-	6	31
Ld F1 0(R1)	4	7	15-20	21	32
Mult.d F2 F1 F0	10	22-26	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	16	23-27	-	30	35
Add.d F4 F2 F4	17	31-33	-	34	36

Appendix 2. Sample test case 02

This input comes from the supplied sample report. Its hardware configuration and instruction stream are retained. Each run changes only the issue width.

Unit	RS entries	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	3	3	-	2
FP multiplier	2	15	-	2
Load/store	4	1	5	2

ROB entries: 128. **Issue widths:** 1, 2, 3 and 4.
Initial registers: R1=12, R2=32, F20=3; all others are zero.
Initial memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4; all other words are zero.

```
 1. Ld F1 0(R1)
 2. Mult.d F2 F1 F0
 3. Add.d F3 F3 F2
 4. Mult.d F2 F1 F1
 5. Ld F1 0(R1)
 6. Addi R1 R1 4
 7. Add.d F4 F2 F4
 8. Addi R1 R1 8
 9. Ld F1 0(R1)
10. Mult.d F2 F1 F0
11. Add.d F3 F3 F2
12. Mult.d F2 F1 F1
13. Ld F1 0(R1)
14. Mult.d F2 F1 F0
15. Add.d F3 F3 F2
16. Mult.d F2 F1 F1
17. Add.d F4 F2 F4
```

Final values at every issue width

Nonzero registers: R1=24, R2=32, F1=6, F2=36, F4=37, F20=3.

Nonzero memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All other register and memory values are zero.

Input: benchmarks/sample_report_02.txt. Complete outputs: results/sample_report_02_w1.txt through sample_report_02_w4.txt.

Case 02: issue widths 1 and 2

EX and MEM intervals include both endpoints. A dash means the stage does not apply. All 17 instructions commit; these sample streams contain no branches.

Issue width 1 | 47 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	2	9-23	-	24	25
Add.d F3 F3 F2	3	25-27	-	28	29
Mult.d F2 F1 F1	4	9-23	-	25	30
Ld F1 0(R1)	5	6	8-12	13	31
Addi R1 R1 4	6	7	-	9	32
Add.d F4 F2 F4	7	26-28	-	29	33
Addi R1 R1 8	8	10	-	11	34
Ld F1 0(R1)	9	12	13-17	18	35
Mult.d F2 F1 F0	10	19-33	-	34	36
Add.d F3 F3 F2	11	35-37	-	38	39
Mult.d F2 F1 F1	12	19-33	-	35	40
Ld F1 0(R1)	13	14	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	26	27-41	-	42	45
Add.d F4 F2 F4	27	43-45	-	46	47

Issue width 2 | 46 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	1	9-23	-	24	25
Add.d F3 F3 F2	2	25-27	-	28	29
Mult.d F2 F1 F1	2	9-23	-	25	30
Ld F1 0(R1)	3	4	8-12	13	31
Addi R1 R1 4	3	4	-	5	32
Add.d F4 F2 F4	4	26-28	-	29	33
Addi R1 R1 8	4	6	-	7	34
Ld F1 0(R1)	5	8	13-17	18	35
Mult.d F2 F1 F0	9	19-33	-	34	36
Add.d F3 F3 F2	9	35-37	-	38	39
Mult.d F2 F1 F1	10	19-33	-	35	40
Ld F1 0(R1)	10	11	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	25	26-40	-	41	45
Add.d F4 F2 F4	26	42-44	-	45	46

Case 02: issue widths 3 and 4

EX and MEM intervals include both endpoints. A dash means the stage does not apply. All 17 instructions commit; these sample streams contain no branches.

Issue width 3 | 46 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	1	9-23	-	24	25
Add.d F3 F3 F2	1	25-27	-	28	29
Mult.d F2 F1 F1	2	9-23	-	25	30
Ld F1 0(R1)	2	3	8-12	13	31
Addi R1 R1 4	2	3	-	4	32
Add.d F4 F2 F4	3	26-28	-	29	33
Addi R1 R1 8	3	5	-	6	34
Ld F1 0(R1)	3	7	13-17	18	35
Mult.d F2 F1 F0	9	19-33	-	34	36
Add.d F3 F3 F2	9	35-37	-	38	39
Mult.d F2 F1 F1	9	19-33	-	35	40
Ld F1 0(R1)	10	11	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	25	26-40	-	41	45
Add.d F4 F2 F4	26	42-44	-	45	46

Issue width 4 | 46 total cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	1	9-23	-	24	25
Add.d F3 F3 F2	1	25-27	-	28	29
Mult.d F2 F1 F1	1	9-23	-	25	30
Ld F1 0(R1)	2	3	8-12	13	31
Addi R1 R1 4	2	3	-	4	32
Add.d F4 F2 F4	2	26-28	-	29	33
Addi R1 R1 8	2	5	-	6	34
Ld F1 0(R1)	3	7	13-17	18	35
Mult.d F2 F1 F0	9	19-33	-	34	36
Add.d F3 F3 F2	9	35-37	-	38	39
Mult.d F2 F1 F1	9	19-33	-	35	40
Ld F1 0(R1)	9	10	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	25	26-40	-	41	45
Add.d F4 F2 F4	26	42-44	-	45	46